Tasks on terraform-01

1. Install Terraform on Windows PC

Objective

To install Terraform on a Windows computer and verify that it works from git bash.

Implementation Steps

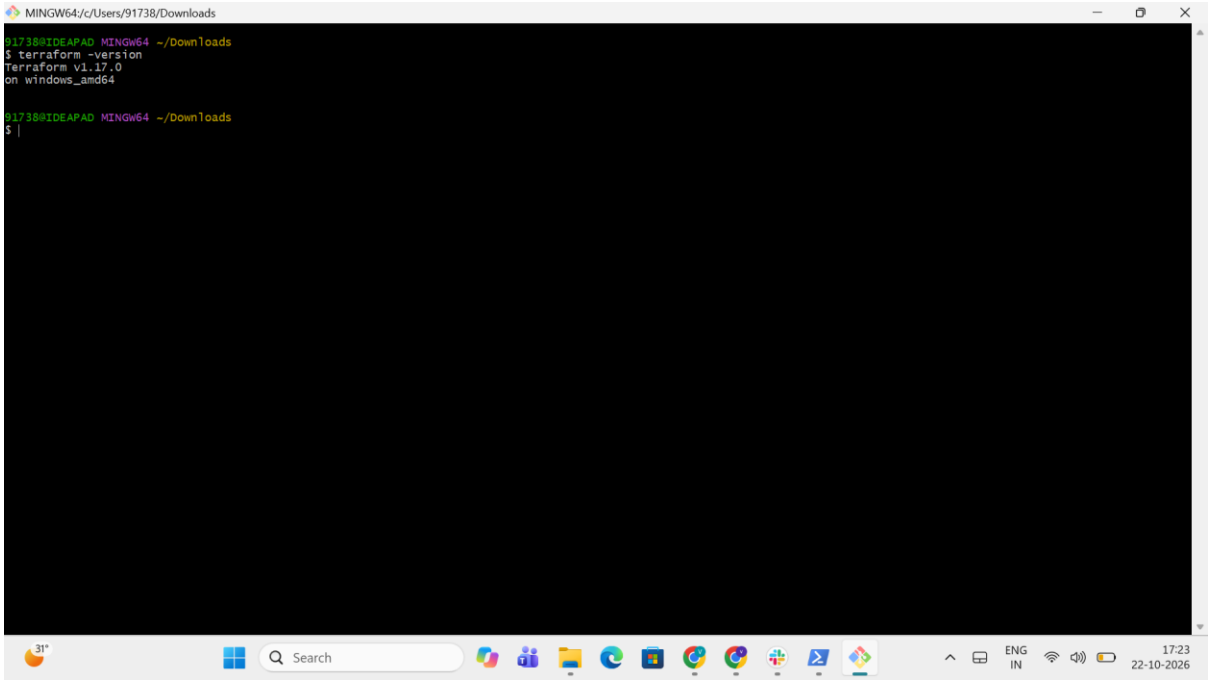
1. Open the official HashiCorp Terraform installation page: https://developer.hashicorp.com/terraform/install
2. Download the Terraform ZIP file for Windows AMD64.
3. Extract the downloaded ZIP file.
4. Create a folder named C:\terraform.
5. Copy terraform.exe into the C:\terraform folder.
6. Open Windows Search and select **Edit the system environment variables**.
7. Click **Environment Variables**.
8. Under User variables, select Path and click **Edit**.
9. Click **New** and add C:\terraform.
10. Save the changes by clicking **OK** on all open windows.
11. Open a new git bash.
12. Execute the following command:

terraform -version

13. Verify that the Terraform version and operating system information are displayed.
14. Install Visual Studio Code and add the HashiCorp Terraform extension to support Terraform syntax highlighting and editing.

Result

Terraform was installed successfully on Windows and verified through git bash.



---

2. Execute the Terraform Templates Demonstrated in the Video

Objective

To create Terraform configuration files, initialize Terraform, preview infrastructure changes, apply the configuration, and understand Terraform state management.

A. Create the Terraform Project

1. Open gitbash.
2. Create a project directory:

   mkdir C:\terraform-basic

   cd C:\terraform-basic
3. Create a file named main.tf:

   main.tf
4. Add the following Terraform configuration:

   resource "local_file" "my_pet" {

    filename = "pets.txt"

    content = "I love pets"

   }
5. Save the file.

B. Initialize Terraform

1. Execute:

   terraform init

2. Terraform initializes the working directory and downloads the required provider plugins.
3. Verify that initialization completes successfully.

C. Preview the Configuration

1. Execute:

   terraform plan

2. Review the proposed changes.
3. Confirm that Terraform plans to create the pets.txt file.

D. Apply the Configuration

1. Execute:

   terraform apply

2. Review the proposed changes.
3. Type yes when prompted.
4. Verify that pets.txt is created in the project directory.
5. Open the file and verify that it contains I love pets.

E. Modify the Existing Configuration

1. Open main.tf.
2. Change the content to:

```
resource "local_file" "my_pet" {
    filename = "pets.txt"
    content  = "I hate pets"
}
```

3. Save the file.
4. Execute:

   terraform plan

5. Review the changes Terraform proposes.
6. Execute:

   terraform apply

7. Type yes when prompted.
8. Open pets.txt and verify the updated content.

F. Work with the Random Pet Provider

1. Add the following configuration to main.tf:

   ```
   resource "random_pet" "my_pet" {
    prefix    = "Mr"
    separator = "."
    length    = 1
   }
   ```

2. Save the file.
3. Initialize the provider:

   terraform init

4. Format the configuration:

   terraform fmt

5. Preview the changes:

   terraform plan

6. Apply the configuration:

   terraform apply

7. Type yes when prompted.
8. Verify the generated pet name in the output.

G. Inspect Terraform State

1. List the resources managed by Terraform:

   terraform state list

2. Inspect the random pet resource:

   terraform state show random_pet.my_pet

3. Observe the resource attributes recorded in Terraform state.

H. Destroy the Resources

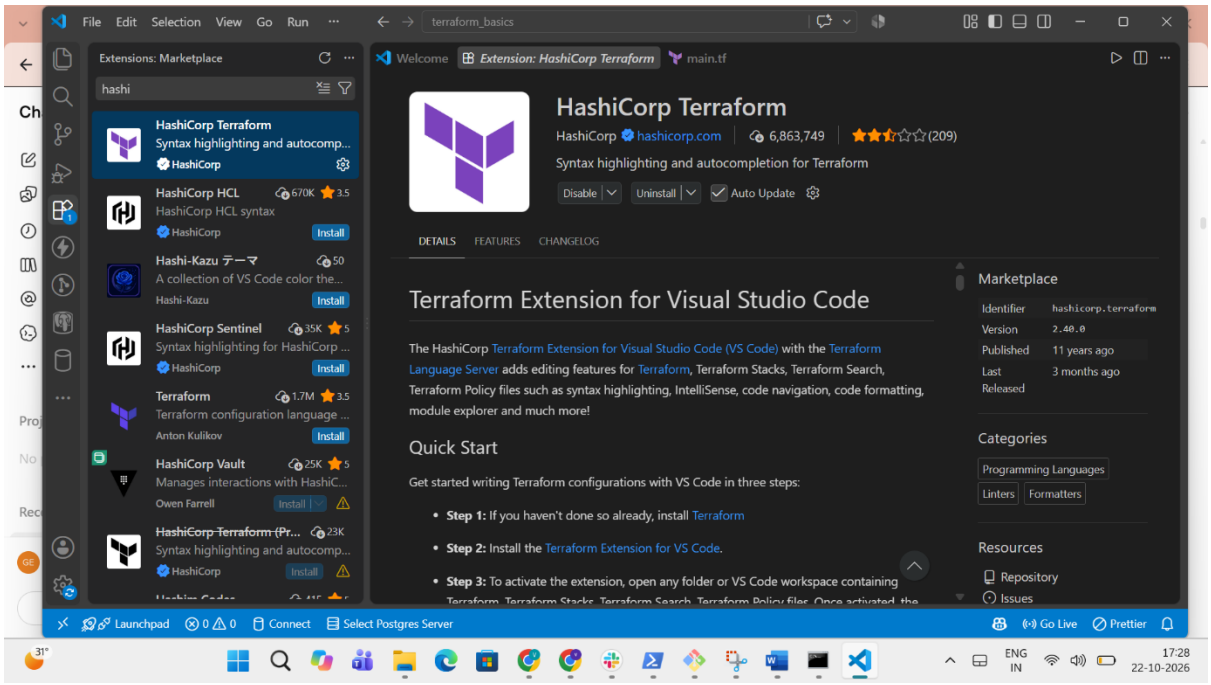
1. Execute:

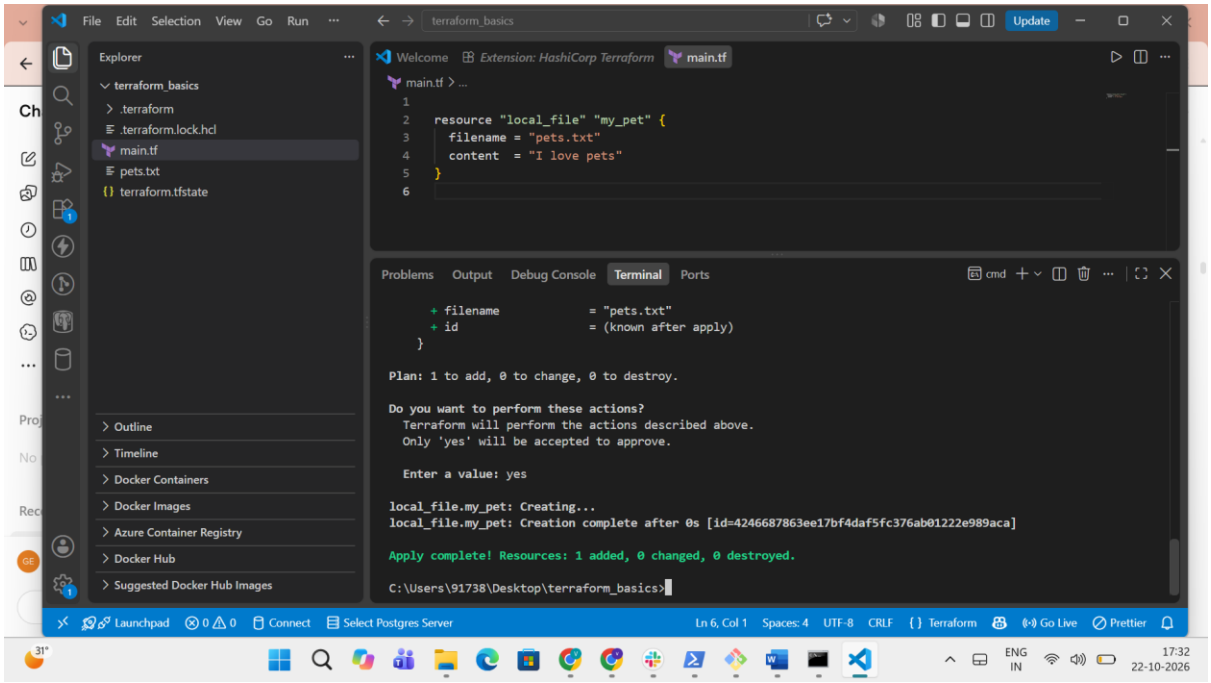
   terraform destroy

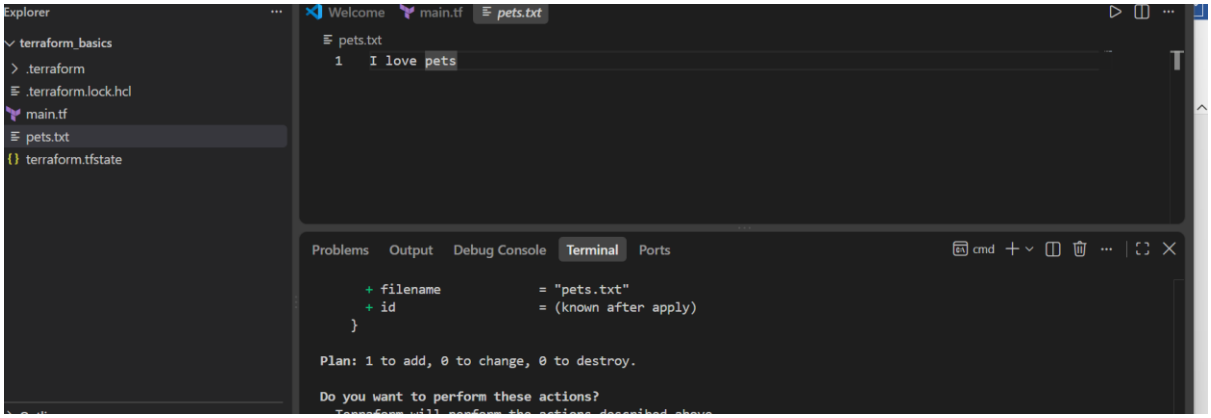
2. Review the resources Terraform proposes to remove.
3. Type yes when prompted.
4. Verify that Terraform removes the managed resources.

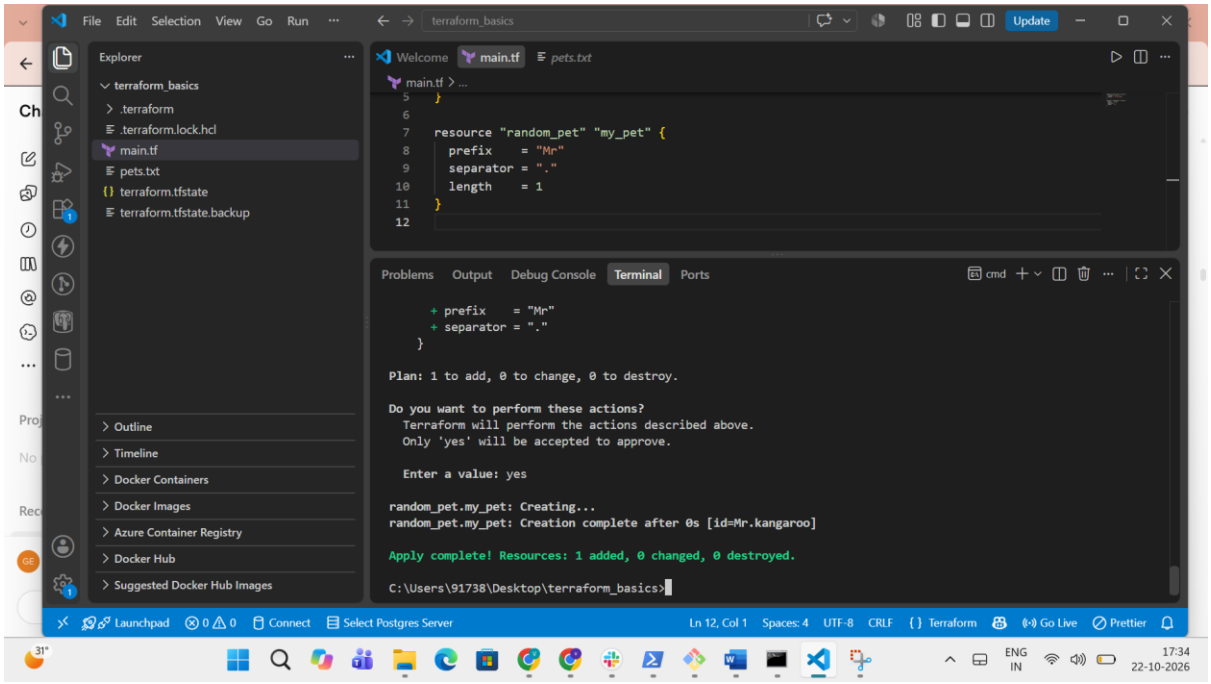
Result

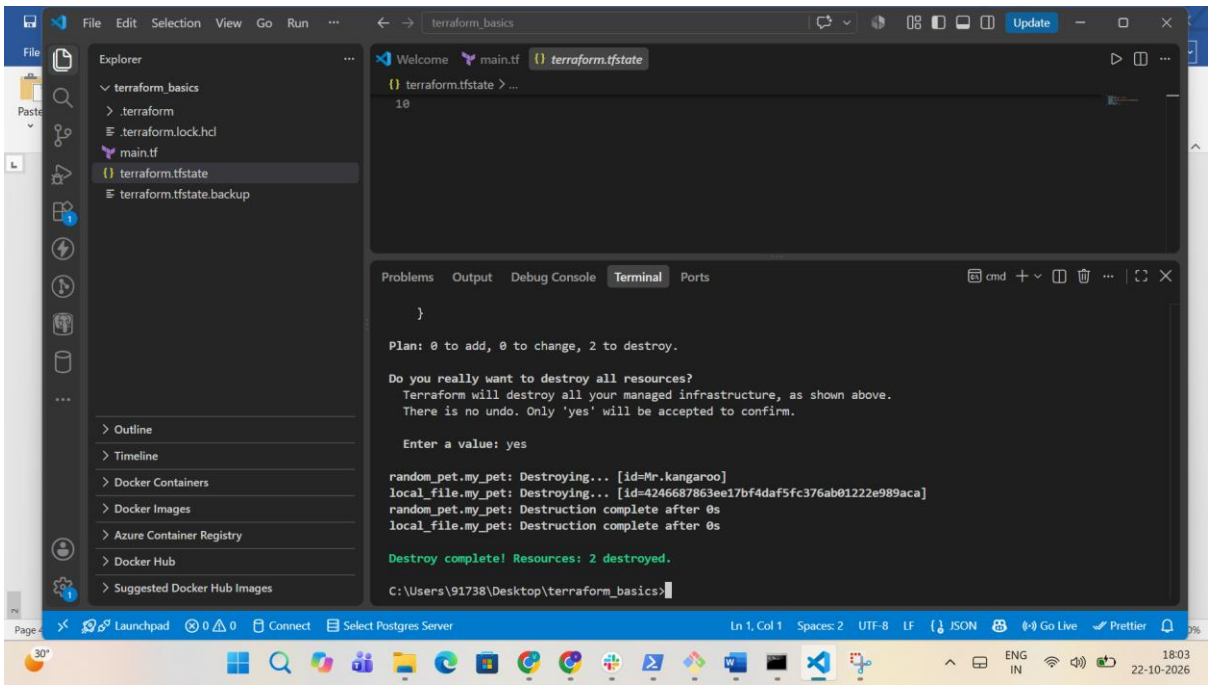
The local file and random pet templates were executed to understand Terraform initialization, planning, applying, formatting, state management, and resource destruction.






Explorer
terraform_basics
.terraform
.terraform.lock.hcl
main.tf
pets.txt
terraform.tfstate
Welcome
main.tf
pets.txt
pets.txt
1 I love pets
Problems
Output
Debug Console
Terminal
Ports
cmd
+ filename = "pets.txt"
+ id = (known after apply)
}
Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?


File Edit Selection View Go Run
terraform_basics
Update
Explorer
terraform_basics
.terraform
.terraform.lock.hcl
main.tf
pets.txt
terraform.tfstate
terraform.tfstate.backup
Welcome
main.tf
pets.txt
resource "random_pet" "my_pet" {
prefix = "Mr"
separator = "."
length = 1
}
Problems
Output
Debug Console
Terminal
Ports
cmd
+ prefix = "Mr"
+ separator = "."
}
Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
random_pet.my_pet: Creating...
random_pet.my_pet: Creation complete after 0s [id=Mr.kangaroo]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
C:\Users\91738\Desktop\terraform_basics>
Outline
Timeline
Docker Containers
Docker Images
Azure Container Registry
Docker Hub
Suggested Docker Hub Images
Launchpad
Connect
Select Postgres Server
Ln 12, Col 1
Spaces: 4
UTF-8
CRLF
Terraform
Go Live
Prettier
ENG
IN
17:34
22-10-2026


File Edit Selection View Go Run
terraform_basics
Update
Explorer
terraform_basics
.terraform
.terraform.lock.hcl
main.tf
terraform.tfstate
terraform.tfstate.backup
Welcome
main.tf
terraform.tfstate
Problems
Output
Debug Console
Terminal
Ports
cmd
}
Plan: 0 to add, 0 to change, 2 to destroy.
Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.
Enter a value: yes
random_pet.my_pet: Destroying... [id=Mr.kangaroo]
local_file.my_pet: Destroying... [id=4246687863ee17bf4daf5fc376ab01222e989aca]
random_pet.my_pet: Destruction complete after 0s
local_file.my_pet: Destruction complete after 0s
Destroy complete! Resources: 2 destroyed.
C:\Users\91738\Desktop\terraform_basics>
Outline
Timeline
Docker Containers
Docker Images
Azure Container Registry
Docker Hub
Suggested Docker Hub Images
Launchpad
Connect
Select Postgres Server
Ln 1, Col 1
Spaces: 2
UTF-8
LF
JSON
Go Live
Prettier
ENG
IN
18:03
22-10-2026

---

3. Note down below points, Terraform Init Terraform Plan Terraform Apply Terraform Provider.

Objective

To understand the main Terraform commands and the role of providers in managing resources.

A. Terraform Init

Purpose: Initializes the Terraform working directory.

B. Terraform Plan

Purpose: Previews the changes Terraform intends to make before applying them.

C. Terraform Apply

Purpose: Applies the planned changes and creates, updates, or deletes resources.

D. Terraform Provider

Purpose: A provider is a plugin that enables Terraform to communicate with and manage resources through a particular platform or service.

Examples:

- local – manages local files.
- random – generates random values.
- aws – manages AWS resources.
- azurerm – manages Azure resources.
- google – manages Google Cloud resources.

---

4. Integrate a Sample Terraform Template with Jenkins on EC2

Objective

To install Terraform on an AWS EC2 instance running Jenkins and execute a Terraform configuration through a Jenkins Pipeline job.

A. Launch and Configure EC2

1. Log in to the AWS Management Console.
2. Open EC2 and launch a new instance.
3. Select Amazon Linux 2023 as the operating system.
4. Select a suitable instance type for the lab.
5. Configure the key pair and network settings.

6. Configure the security group to allow SSH on port 22 from your IP address.
7. Allow Jenkins access on port 8080 from your approved IP range.
8. Launch the instance and wait until it reaches the Running state.

B. Install Java and Jenkins

1. Connect to the EC2 instance using EC2 Instance Connect or SSH.
2. Update the system packages:

   sudo dnf update -y
3. Install Java:

   sudo dnf install -y java-21-amazon-corretto fontconfig
4. Add the Jenkins stable repository:

   sudo wget -O /etc/yum.repos.d/jenkins.repo \

   https://pkg.jenkins.io/rpm-stable/jenkins.repo
5. Import the Jenkins repository signing key:

   sudo rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key
6. Install Jenkins:

   sudo dnf install -y jenkins
7. Enable and start Jenkins:

   sudo systemctl enable jenkins

   sudo systemctl start jenkins
8. Verify the service:

   sudo systemctl status jenkins
9. Retrieve the initial administrator password:

   sudo cat /var/lib/jenkins/secrets/initialAdminPassword
10. Open Jenkins in a browser using:

    http://<EC2-PUBLIC-IP>:8080
11. Enter the initial password, install the suggested plugins, and create the administrator account.

C. Install Terraform on EC2

1. Install the repository management utility:

   sudo dnf install -y dnf-plugins-core
2. Add the HashiCorp repository:

sudo dnf config-manager --add-repo \

https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo

3. Install Terraform:

   sudo dnf install -y terraform

4. Verify the installation:

   terraform -version

   which terraform

5. Verify that the Jenkins execution user can access Terraform:

   sudo -u jenkins terraform -version

D. Create the Jenkins Pipeline Job

1. Open the Jenkins dashboard.
2. Click **New Item**.
3. Enter the job name terraform-demo.
4. Select **Pipeline**.
5. Click **OK**.
6. Scroll down to the Pipeline section.
7. Select **Pipeline script** under Definition.
8. Add the following sample pipeline script:

```
pipeline {
    agent any

    stages {
        stage('Check Terraform') {
            steps {
                sh 'terraform -version'
            }
        }

        stage('Create Terraform Template') {
            steps {
                writeFile file: 'main.tf', text: '''
```

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.0"
    }
  }
}

resource "local_file" "jenkins_demo" {
  filename = "${path.module}/jenkins-output.txt"
  content  = "Terraform executed successfully through Jenkins"
}
'''
            }
        }

        stage('Terraform Init') {
            steps {
                sh 'terraform init -input=false'
            }
        }

        stage('Terraform Validate') {
            steps {
                sh 'terraform fmt -check'
                sh 'terraform validate'
            }
        }

        stage('Terraform Plan') {
```

```
            steps {
                sh 'terraform plan -input=false -out=tfplan'
            }
        }

        stage('Terraform Apply') {
            steps {
                sh 'terraform apply -input=false -auto-approve tfplan'
            }
        }

        stage('Verify Output') {
            steps {
                sh 'cat jenkins-output.txt'
            }
        }
    }
}
```

9. Click **Save**.

E. Execute the Jenkins Job

1. Open the terraform-demo job.
2. Click **Build Now**.
3. Select the latest build number.
4. Open **Console Output**.
5. Verify that Terraform initializes, validates, plans, and applies the configuration.
6. Confirm that the output contains:

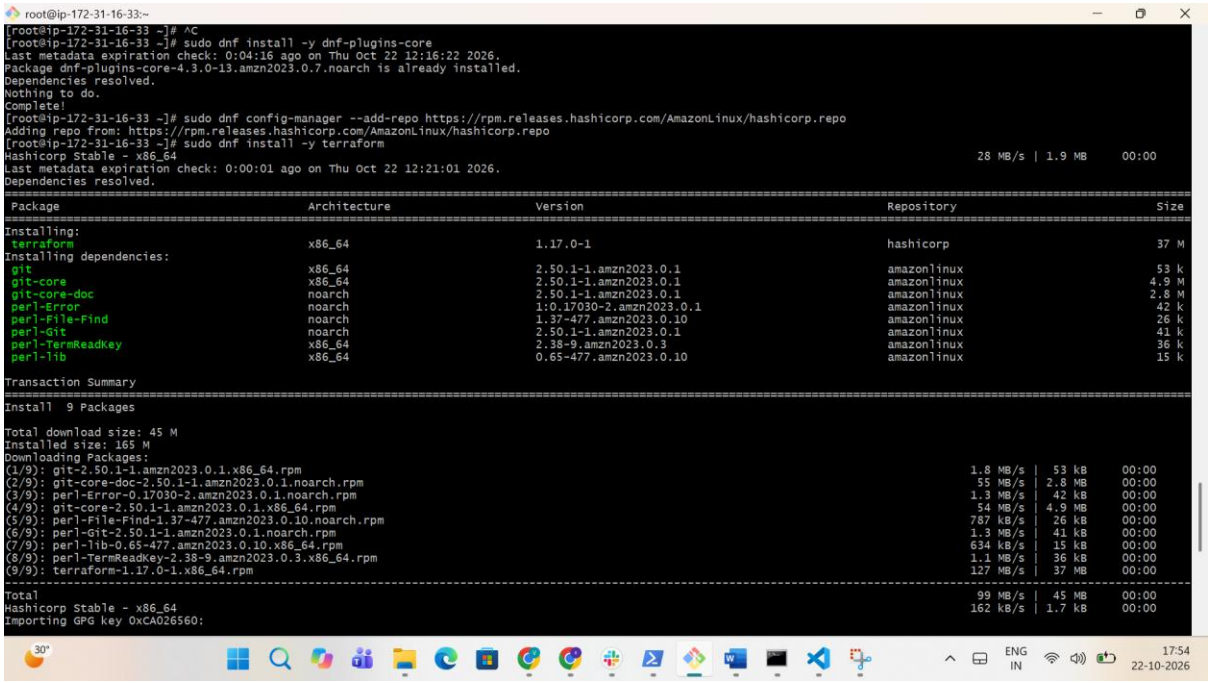
Terraform executed successfully through Jenkins

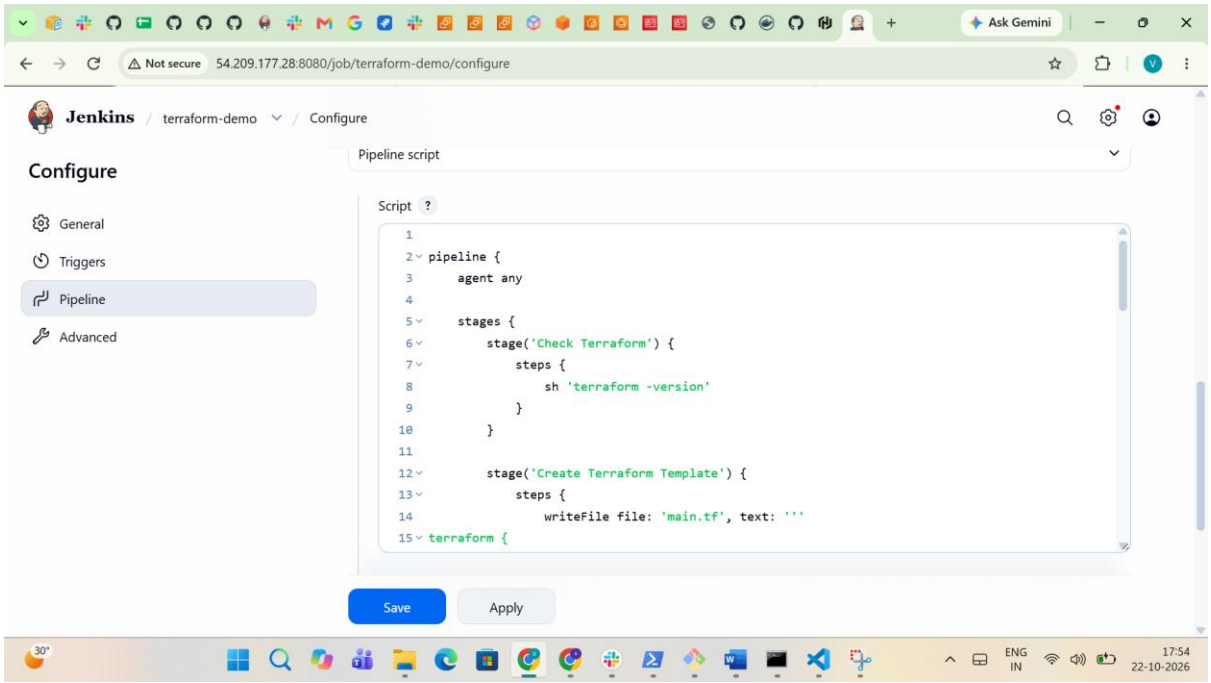
7. Verify that the job finishes with:

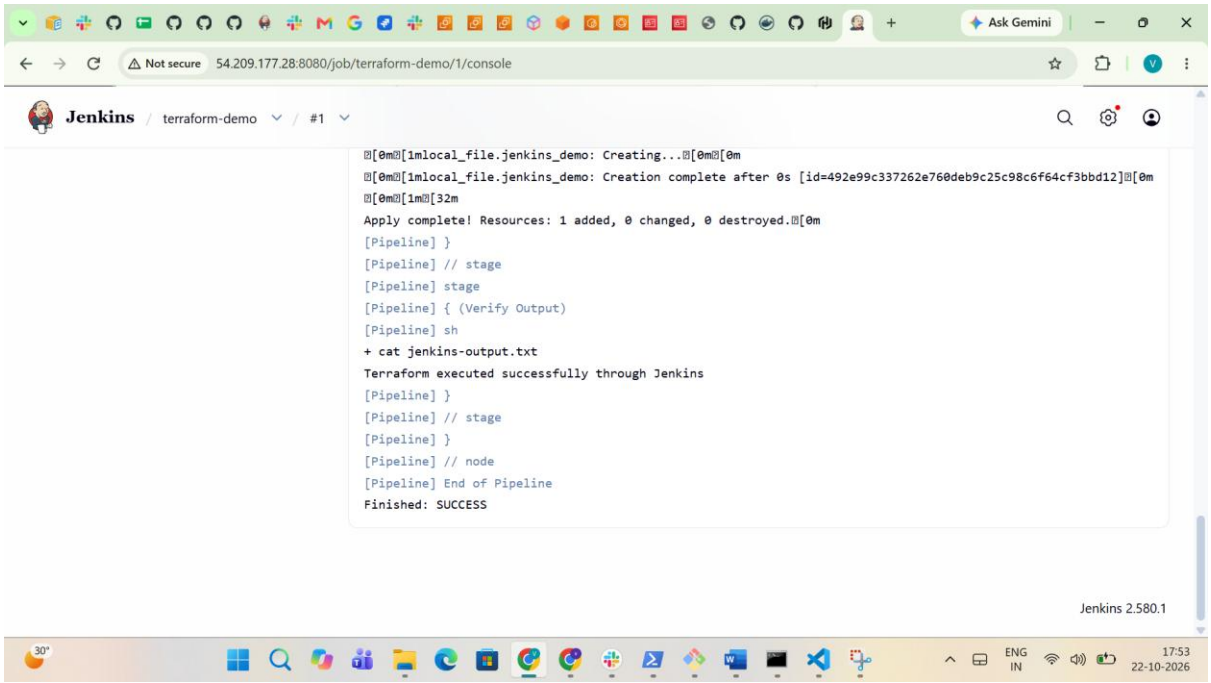
Finished: SUCCESS

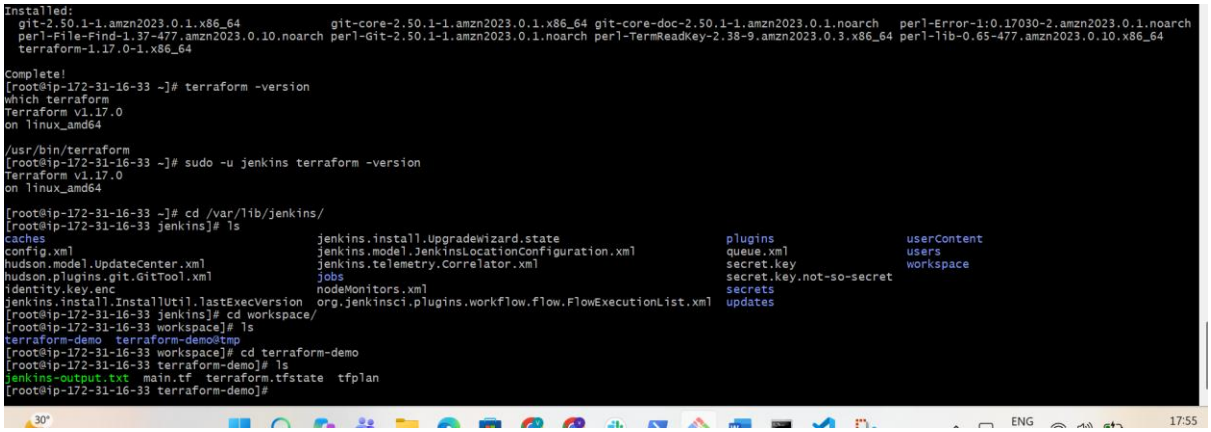
Result

Terraform was integrated with Jenkins on EC2, and a sample Terraform template was executed through a Jenkins Pipeline. The pipeline created a local output file in the Jenkins workspace.






Ask Gemini
Not secure
54.209.177.28:8080/job/terraform-demo/1/console
Jenkins
terraform-demo
#1
[0m[1mlocal_file.jenkins_demo: Creating...[0m[0m
[0m[1mlocal_file.jenkins_demo: Creation complete after 0s [id=492e99c337262e760deb9c25c98c6f64cf3bbd12][0m
[0m[1m[32m
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.[0m
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Verify Output)
[Pipeline] sh
+ cat jenkins-output.txt
Terraform executed successfully through Jenkins
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
Jenkins 2.580.1
ENG
IN
17:53
22-10-2026


Installed:
git-2.50.1-1.amzn2023.0.1.x86_64 git-core-2.50.1-1.amzn2023.0.1.x86_64 git-core-doc-2.50.1-1.amzn2023.0.1.noarch perl-Error-1:0.17030-2.amzn2023.0.1.noarch
perl-File-Find-1.37-477.amzn2023.0.10.noarch perl-Git-2.50.1-1.amzn2023.0.1.noarch perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64 perl-lib-0.65-477.amzn2023.0.10.x86_64
terraform-1.17.0-1.x86_64
Complete!
[root@ip-172-31-16-33 ~]# terraform -version
which terraform
Terraform v1.17.0
on linux_amd64
/usr/bin/terraform
[root@ip-172-31-16-33 ~]# sudo -u jenkins terraform -version
Terraform v1.17.0
on linux_amd64
[root@ip-172-31-16-33 ~]# cd /var/lib/jenkins/
[root@ip-172-31-16-33 jenkins]# ls
caches jenkins.install.UpgradeWizard.state plugins userContent
config.xml jenkins.model.JenkinsLocationConfiguration.xml queue.xml users
hudson.model.UpdateCenter.xml jenkins.telemetry.Correlator.xml secret.key workspace
hudson.plugins.git.GitTool.xml jobs secret.key.not-so-secret
identity.key.enc nodeMonitors.xml secrets
jenkins.install.InstallUtil.lastExecVersion org.jenkinsci.plugins.workflow.flow.FlowExecutionList.xml updates
[root@ip-172-31-16-33 jenkins]# cd workspace/
[root@ip-172-31-16-33 workspace]# ls
terraform-demo terraform-demo@tmp
[root@ip-172-31-16-33 workspace]# cd terraform-demo
[root@ip-172-31-16-33 terraform-demo]# ls
jenkins-output.txt main.tf terraform.tfstate tfplan
[root@ip-172-31-16-33 terraform-demo]#
ENG
17:55